ARCHITECTURE FOR SCALE

GGG.Cafe · Systems review · November 2026

Architecture review, 2026-11-12. It covers where the system stands, the target design, and the first step built.

1. SUMMARY

GGG.Cafe is a Next.js 16 app on Vercel over Supabase (Postgres, Auth, Storage). The shape is right for the product:

- Stateless servers that scale out on their own.
- One relational database, with row-level security (RLS) as the single authorization boundary.
- Private file storage.
- A small number of outside services.

The site doesn't need a rewrite to grow. It needs six things, in this order:

1. **Protect the database.** Rate limits on costly and busy writes. Built this round.
2. **An async job layer:** an outbox table and a scheduled worker, for email, retries and cleanup.
3. **Observability:** error tracking, structured action logs and query insights, before traffic arrives.
4. **Tenancy:** an organizations layer, so one deployment serves many clubs.
5. **Shared caching:** team-wide reference data, tagged and invalidated by the action that changes it.
6. **Database headroom:** compute size, then a read replica, then partitioning of the fastest-growing tables.

2. WHERE IT STANDS (MEASURED 2026-11-12)

AREA	TODAY
Web	Next.js 16.4 (App Router, Cache Components, React 19.3) on Vercel
Data	Supabase Postgres 17, US East (Ohio); 60 max connections; pg_cron
Schema	92 migrations, 132 tables, about 270 RLS policies, about 231 SQL functions, 114 indexes
App	93 pages, 4 route handlers, 232 server actions in 50 files, 90 client components
Code	about 62,000 lines of TypeScript, about 17,500 lines of SQL
Live data	1 team, 11 players, 2 accounts, 23 MB database, 1 stored file
Outside services	Zoom (Server-to-Server OAuth and Meeting SDK), Resend (optional email)
Checks on every push	lint, types, 204 unit tests, database tests, build, 105-page smoke test, accessibility

What scales well already:

- Rendering is stateless.
- Authorization lives in the database, so every path is covered (RLS with helper sets read once per query).
- Independent queries run together.
- Foreign keys are indexed.
- Private files use signed links that expire.

Limits:

- One club per deployment: staff are global and teams have no tenant boundary.
- Every page read reaches Postgres; nothing is shared across users or requests.
- Work runs inside the request: notifications by trigger, Zoom calls inline, no email outbox.
- Until this round, no limit on how fast a signed-in user could write.
- Errors show only in Vercel's logs.

3. SCALE TARGETS AND LOAD MODEL

PHASE	USERS	PEAK CONCURRENT	PAGE VIEWS AT PEAK	DATABASE QUERIES AT PEAK
1. One full club (20 teams)	about 1,000	150 (game day)	about 8/s	about 80/s
2. 100 clubs	about 100,000	5,000	about 250/s	about 2,500/s
3. 1,000 clubs	about 1,000,000	40,000	about 2,000/s	about 20,000/s

Assumptions:

- A page view is one function run, with 5 to 12 database API queries in one or two rounds.
- An active user opens a page about every 20 seconds.
- Writes are under 10% of requests.

What breaks first, in order:

1. Postgres CPU and connections.
2. Calls to Supabase's Storage and Auth APIs (signing photo links, reading the session).
3. Third-party quotas: Zoom API calls, email volume.

Vercel scales out on its own and isn't the constraint.

4. TARGET ARCHITECTURE

CLIENTS
Browser and the installable phone app (PWA). Zoom's web kit runs in its own frame on the call page.

↓ HTTPS

VERCEL EDGE
CDN: static shell, assets, public pages. proxy.ts: refreshes the session, sends people to agree to new Terms.

↓

NEXT.JS SERVER (STATELESS, SCALES OUT)
Server Components read behind Suspense. Server actions write (FormState). Route handlers: PDFs, auth links, webhooks and cron (proposed).

↓

DATA ACCESS LAYER: SRC/LIB
auth guards, loaders, pure rules (unit-tested), zoom.ts, email.ts

CACHE
Browser: use cache: private. Server: use cache: remote, tagged per team (proposed). Per request: React cache.

OUTSIDE SERVICES
Zoom (S2S OAuth, Meeting SDK), Resend email, Stripe for G3 (proposed)

↓ user's token (JWT); RLS decides

SUPABASE
Postgres 17: RLS on every table, triggers (notifications, rate limits), definer functions, pg_cron. Auth. Storage: private buckets, signed links.

↓

ASYNC (PROPOSED)
Outbox jobs table, then a Vercel Cron worker: email, retries, cleanup

OBSERVABILITY (PROPOSED)
Error tracking, structured action logs, log drain, query insights

PHASE 3 (PROPOSED)
Read replica, monthly partitions, queue service

LAYER	ROLE	SCALE LEVER
Edge (Vercel CDN, proxy.ts)	Static shell, assets, public pages; refreshes the session; sends people to agree to new Terms	Cache-Control on public pages
App (Next.js server)	Server Components read; server actions write; route handlers serve PDFs, webhooks and cron	Stateless, scales out
Data access (src/lib)	One place per concern: auth.ts, roster.ts, zoom.ts; pure rules in no-import modules	Shared caching sits here
Database (Postgres with RLS)	Source of truth and the authorization boundary; triggers, definer functions	Compute size, replica, partitions
Files (Storage)	Private buckets: photos, voice memos, consent forms, logos	Signed links reused per page
Async (pg_cron, outbox, worker)	Scheduled jobs; email, retries and cleanup off the request path	Batch size, schedule, a queue service later
Outside services	Zoom, email, payments later	Rate limits, retries, webhooks
Observability	Logs, errors, query statistics	Log drain, error tracking

Source: docs/architecture/scaling.md

5. COMPONENT STRUCTURE

```
src/app/                    routes: public (/, /login, /terms), (staff)/ (coaches; player pages),
                            family/, mentor/, calls/ (video calls); feature folders hold page.tsx,
                            actions.ts and client forms
src/lib/                    the data access layer and rules
  auth.ts                   who is signed in, and the page and action guards
  roster.ts, dugout data    shared loaders (one query shape per concern)
  video-calls.ts, journal.ts, switchboard.ts ...   pure rules: no imports, unit-tested
  zoom.ts, email.ts         outside services (server-only)
  supabase/                 server and browser clients; generated types; sample data for tests
src/components/             shared UI: ui.tsx (forms, badges, cards), notification bell, video calls
supabase/migrations/        the schema, applied in order; definer functions in the private schema
                            with public wrappers made by private.expose()
tests/unit, tests/db        rules and database checks; scripts/ run the page and accessibility checks
```

Principles:

- **Rules live in pure modules.** The pages, the actions and the tests share them.
- **Authorization is checked twice:** in the action, which gives a fast, friendly message, and in the database, which is the real boundary.
- **Every write reports back** through one contract (FormState).
- **The schema is the source of the types** (npm run db:types). Literal select strings keep results typed.

6. DATA FLOW

Read (a page):

1. The browser requests a page. proxy.ts refreshes the Supabase session cookie and checks the Terms version.
2. The static shell streams at once. Everything that reads the session sits behind a Suspense boundary.
3. Loaders in src/lib build a Supabase client with the user's token and run independent queries together.
4. Postgres applies RLS: helper sets such as my_player_ids() and my_child_ids() are evaluated once per query. Rows come back, Server Components render, and the result streams to the browser.

Write (a server action):

1. A form posts to a server action. It checks who's acting (auth.ts guards).
2. It validates with a pure rule (for example cleanCallInput), then writes through Supabase with must().
3. Postgres checks RLS and runs triggers: rate limits, notifications, derived columns.
4. The action calls refresh(), and the page re-renders with fresh data. Errors come back as one friendly message.

Video call (the most involved flow):

1. A mentor books a call. The action checks the rate limit, then asks Zoom for a meeting (Server-to-Server OAuth).
2. The action saves the call. The database checks who may book, one call at a time, and the limit, and a trigger notifies her parents and her login.
3. At call time, joinVideoCall checks the person's role through a definer function, signs a Meeting SDK token, and gets the host key for the host only.
4. The page runs Zoom's web kit in a separate frame (its own React) and hands it the join details by postMessage, same origin only.

Async (proposed):

1. An action writes its row and a job row in the same transaction.
2. A worker on Vercel Cron claims jobs with `for update skip locked`, runs them (email, Zoom cleanup), and records success or the error with a retry time.
3. Jobs are idempotent, keyed by kind and subject.

7. API DESIGN

Internal API: server actions. These are the only way the site writes.

- Form actions: `(prev: FormState, fd: FormData) => Promise<FormState>`.
- Object actions: `(input) => Promise<{ error }>`.
- Each one authorizes, validates with a pure rule, writes, then calls refresh().
- Never trust the client: guards run in every action, and RLS runs again in the database.

Database API: PostgREST and RPC.

- Tables are reachable under RLS with the user's token.
- Functions are exposed only through private.expose(), signed-in only by default.
- The publishable key is public, so anyone signed in can call the database API directly. That's why rate limits and validation also live in triggers and constraints, not only in actions.

Error model:

CODE	MEANING	SHOWN AS
42501	Not allowed (RLS or a guard)	"You don't have permission…"
23505 / 23514 / 23503	Duplicate / out of range / missing reference	A specific message
P0006	Domain rule: not on our roster	A specific message
P0007 / P0008	Video call in the past / time already booked	Marked on the time field
P0009	Rate limit reached	"That's a lot in a short time…"

HTTP endpoints (route handlers):

ENDPOINT	USE	AUTH
GET /dugout/*/pdf, consent form	PDF exports	Session; RLS on the data
GET /auth/confirm	Sign-in links (code or token)	One-time token
GET /api/cron/jobs (proposed)	Outbox worker	Bearer CRON_SECRET (Vercel Cron)
POST /api/webhooks/zoom (proposed)	Meeting ended, participant joined (call log)	Zoom signature header
POST /api/webhooks/stripe (proposed)	G3 subscriptions (source = subscription)	Stripe signature, idempotency key
GET /api/v1/* (later)	Partner read API (schedules, rosters)	Organization API token, versioned

8. DATABASE SCHEMA

The core model:

- teams, then seasons, then roster_entries, which point to players.
- Who sees what comes from access edges:
  - staff (role: viewer, assistant, coach, admin)
  - team_admins (parents running a team's calendar and GameDay)
  - parent_links (parent to daughter, approved by the head coach)
  - player_claims (a login claiming a roster spot)
  - mentor_links (mentor to player or team)

- Feature tables hang off player_id or team_id: goals, journal, measurements, games and box-score lines, video_calls, notifications.

RLS model:

- Identity (auth.uid()) leads to access sets: my_player_ids, my_child_ids, my_mentee_ids, my_admin_team_ids, my_current_team_ids. Policies test membership in those sets.
- Every player-read rule has a parent-view twin.
- Rows that must cross those lines (team leaderboards, rosters for team admins) go through definer functions that check the caller first.

Changes for scale:

- **Tenancy (Phase 2):**
  - New tables: organizations and organization_members.
  - teams.organization_id.
  - Staff roles move from global to per organization.
  - Every staff policy becomes "team in my organization's teams".
- **Outbox (Phase 1):** private.jobs (kind, subject, payload, run_at, attempts, locked_at, done_at, last_error), with a partial index on run_at where the job isn't done.
- **Rate limits (built):** private.rate_limit_hits (user, bucket, window_start, hits). Counters older than a day are purged hourly.
- **Growth tables:** partition by month when a table passes about 10 million rows: notifications, iq_answers, measurements, plate_appearances and pitches. Keep foreign-key and (team, date) indexes.

9. CACHING STRATEGY

LAYER	WHAT	KEY AND INVALIDATION	LIFETIME
CDN	Public pages, assets, code-derived content (lessons, book chapters, legal text)	Build ID	Until the next deploy
Browser (`use cache: private`)	Session-derived header data: role, team logo, menu counts	Per session; refreshed on navigation after stale time	30 s to 5 min
Server shared (`use cache: remote`)	Team-wide reference data: standards, culture, drill library, hitting philosophy, IQ question bank, metric definitions	Tag `team:{id}:{area}`; updateTag in the action that writes it	Hours
Per request (React cache)	Current user, mentor access, approvals count	One request	One request
Database	Season stat tables, leaderboard tallies, materialized summaries	Refreshed on write or by a job	Until the next change

Rules:

- **Personal data never goes in a shared server cache:** medical cards, messages, journals, anything a user sees through their own access sets. It's read per request under RLS.
- **Shared server caching needs a read path without cookies.** Cache scopes can't read the session. So authorize first with the user's session (cheap), then read the team-wide rows through a narrow, server-only reader credential: a Postgres role with SELECT on shared tables only. Key the cache by team and tag it.
- **On Vercel the default in-memory cache rarely survives between requests.** A real hit rate needs a remote cache handler.
- **Keys and tags are stored in plain text.** Use ids only, never emails or tokens.

10. BUILT THIS ROUND: RATE LIMITS

Database (migration 20261113041701, test tests/db/rate_limits.test.sql):

- Fixed-window counters per user and bucket, in the private schema.
- A public check for server actions.
- Triggers on the busy write tables. They hold even for writes that skip the site.

```
-- count one hit for the caller in this window; true while within the limit
create function private.rate_limit_take(p_bucket text, p_max integer, p_window_seconds integer) returns boolean
language plpgsql volatile security definer set search_path = '' as $$
declare v_me uuid := (select auth.uid()); v_start timestamptz; v_hits integer;
begin
  if v_me is null then raise exception 'not allowed' using errcode = '42501'; end if;
  v_start := to_timestamp(floor(extract(epoch from now()) / p_window_seconds) * p_window_seconds);
  insert into private.rate_limit_hits as h (user_id, bucket, window_start, hits)
  values (v_me, p_bucket, v_start, 1)
  on conflict (user_id, bucket, window_start) do update set hits = h.hits + 1
  returning h.hits into v_hits;
  return v_hits <= p_max;
end $$;

-- triggers: 60 mentor messages, 30 journal entries, 12 video call bookings per hour per person
create trigger rate_limit before insert on public.mentor_messages
  for each row execute function private.limit_mentor_messages();
```

App:

- Server actions check before each call to Zoom (12 bookings an hour, 30 joins every 10 minutes).
- friendlyDbError turns P0009 into one plain message.

```
// src/app/calls/actions.ts: before asking Zoom for a meeting
const { data: allowed } = await supabase.rpc("rate_limit_hit", { p_bucket: "zoom.book", p_max: 12, p_window_seconds:
3600 });
if (!allowed) return failed("That's a lot of bookings in a short time. Wait a few minutes and try again.");
```

11. NEXT INCREMENTS (DESIGN CODE)

Outbox worker (Phase 1). Jobs are claimed safely by many workers and retried with backoff.

```
create table private.jobs (
  id bigint generated always as identity primary key,
  kind text not null, subject uuid, payload jsonb not null default '{}',
  run_at timestamptz not null default now(), attempts int not null default 0,
  locked_at timestamptz, done_at timestamptz, last_error text,
  unique (kind, subject)
);
create index jobs_due on private.jobs (run_at) where done_at is null;

create function private.claim_jobs(p_limit int) returns setof private.jobs
language sql security definer set search_path = '' as $$
  update private.jobs j set locked_at = now(), attempts = attempts + 1
   where j.id in (select id from private.jobs where done_at is null and run_at <= now()
                    and (locked_at is null or locked_at < now() - interval '5 minutes')
                  order by run_at limit p_limit for update skip locked)
  returning j.*
$$;
```

```
// src/app/api/cron/jobs/route.ts: Vercel Cron calls this every minute
export async function GET(request: Request) {
  if (request.headers.get("authorization") !== `Bearer ${process.env.CRON_SECRET}`) return new Response(null, {
status: 401 });
  const db = workerClient(); // server-only credential for the jobs functions only
  const { data: jobs } = await db.rpc("claim_jobs", { p_limit: 25 });
```

```
  for (const job of jobs ?? []) {
    try { await runJob(job); await db.rpc("finish_job", { p_id: job.id }); }
    catch (e) { await db.rpc("fail_job", { p_id: job.id, p_error: String(e), p_retry_in: 60 * 2 ** job.attempts }); }
  }
  return Response.json({ ran: jobs?.length ?? 0 });
}
```

Shared cache for team-wide data (Phase 1-2). Authorize per request, read once per team.

```
import "server-only";
import { cacheLife, cacheTag } from "next/cache";

export async function teamStandards(teamId: string) {
  await requireTeamAccess(teamId);          // the user's session, under RLS (cheap)
  return cachedTeamStandards(teamId);
}

async function cachedTeamStandards(teamId: string) {
  "use cache: remote";
  cacheTag(`team:${teamId}:standards`);
  cacheLife("hours");
  const { data } = await readerClient()     // read-only role: SELECT on shared tables only
    .from("team_standards").select("id, title, body, position").eq("team_id", teamId).order("position");
  return data ?? [];
}
// in the action that edits standards: updateTag(`team:${teamId}:standards`)
```

Tenancy (Phase 2). One deployment, many clubs.

```
create table public.organizations (
  id uuid primary key default gen_random_uuid(),
  name text not null, slug text unique not null check (slug ~ '^[a-z0-9-]{3,40}$'),
  plan text not null default 'standard', created_at timestamptz not null default now()
);
create table public.organization_members (
  organization_id uuid not null references public.organizations on delete cascade,
  user_id uuid not null references auth.users on delete cascade,
  role public.staff_role not null,
  primary key (organization_id, user_id)
);
alter table public.teams add column organization_id uuid references public.organizations;
-- backfill today's club as the first organization, then: alter column organization_id set not null

create function private.my_org_team_ids() returns setof uuid
language sql stable security definer set search_path = '' as $$
  select t.id from public.teams t
    join public.organization_members m on m.organization_id = t.organization_id
   where m.user_id = (select auth.uid())
$$;
-- staff policies change from "is_staff()" to "team_id in (select private.my_org_team_ids())"
```

12. ROADMAP

PHASE	TRIGGER	WORK
1. One full club	Now	Rate limits (done). Error tracking and a log drain. Outbox and worker; parent emails for video calls. Private cache for the header. Watch connections and CPU; upgrade compute past 60% CPU or 40 connections.
2. Second club	First outside club signs on	Organizations tenancy and staff per organization. Stripe for G3 subscriptions. Remote cache for team-wide data. Staging project with seeded data. Backups and restore drill.

3. National	Over 2,500 queries/s or a 50 GB database	Read replica for leaderboards, stats and reports. Monthly partitions on growth tables. A queue service for jobs. Usage-based limits per organization.

Risks:

- **RLS cost grows with policy count.** Keep helper sets in `(select ...)` so they run once per query, and add an index for every column a policy filters on.
- **Tenancy is the riskiest migration**, because it touches every staff policy. Do it behind tests that prove no organization can read another's rows.
- **Third-party limits:** Zoom licenses mean one call at a time per host user, and email volume needs custom SMTP. Plan the hosts and senders before the second club.